



PolyMinHash: Efficient Area-Based MinHashing of Polygons for Approximate Nearest Neighbor Search

Alima Subedi
asbmr@mst.edu
Missouri S&T
Rolla, Missouri, USA

Sankalpa Pokharel
sp5rt@mst.edu
Missouri S&T
Rolla, Missouri, USA

Satish Puri
satish.puri@mst.edu
Missouri S&T
Rolla, Missouri, USA

Abstract

Similarity searches are a critical task in data mining. As datasets grow larger, exact nearest neighbor searches quickly become unfeasible, leading to the adoption of approximate nearest neighbor (ANN) searches. ANN has been studied for text data, images, and trajectories. However, there has been little effort to develop ANN systems for polygons in spatial database systems and geographic information systems. We present PolyMinHash, a system for approximate polygon similarity search that adapts MinHashing into a novel 2D polygon-hashing scheme to generate short, similarity-preserving signatures of input polygons. Minhash is generated by counting the number of randomly sampled points needed before the sampled point lands within the polygon's interior area, yielding hash values that preserve area-based Jaccard similarity.

We present the trade-off between search accuracy and runtime of our PolyMinHash system. Our hashing mechanism reduces the number of candidates to be processed in the query refinement phase by up to 98% compared to the number of candidates processed by the Brute Force (Exact Search) algorithm.¹

CCS Concepts

• **Theory of computation** → **Hash functions**; • **Information systems** → *Nearest-neighbor search*.

Keywords

nearest neighbor, MinHashing, locality sensitive hashing, Jaccard distance, polygons, similarity search

ACM Reference Format:

Alima Subedi, Sankalpa Pokharel, and Satish Puri. 2025. PolyMinHash: Efficient Area-Based MinHashing of Polygons for Approximate Nearest Neighbor Search. In *The 33rd ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL '25)*, November 3–6, 2025, Minneapolis, MN, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3748636.3762767>

¹This work is partially supported by NSF grants #2344585, #2344578 and Taylor Geospatial Institute managed by Saint Louis University.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGSPATIAL '25, Minneapolis, MN, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2086-4/2025/11
<https://doi.org/10.1145/3748636.3762767>

1 Introduction

Similarity search plays a crucial role in data mining. The ability to tell if two objects are similar and identify whether an object is closest to a query object is vital in some fundamental algorithms in data mining, such as clustering, information retrieval, and recommender systems. Similarity searches are applied in multiple fields, such as pathology, solar flares [7], and Geographic Information Systems (GIS), including geospatial intelligence, for analyzing the geometries (shapes) of polygons and trajectories. In GIS, the geometries are represented as a list of coordinates (latitude, longitude). An example of shape similarity from the GIS domain is finding a lake that is similar in shape and area to Lake Michigan. In solar physics, to predict solar flares, the query object and the dataset comprise polygons that represent solar events [7]. In digital pathology [9], tissues are represented as polygons for tumor diagnosis, and the Jaccard distance (ratio of intersection to union areas) is used to make similarity comparisons.

As the size of datasets grows, the ability to perform these searches quickly and efficiently becomes increasingly critical to the usability of the system in practical applications. The Brute-Force approach requires scanning the entire dataset to compare it to the query object, which is a linear-time operation per query. For polygonal data, comparing the query polygon with one of the input polygons in the database using the Jaccard similarity $\mathcal{J}_s$ requires expensive computational geometry operations to compute the geometric intersection and union. To find exact nearest neighbors for a query geometry, the geometric Jaccard distance is used, which is derived from geometric similarity ($1 - \mathcal{J}_s$). Using this distance metric, any two similar polygons have a small distance between them. An example of ANN search is given in Fig. 1.

Our proposed PolyMinHash system performs a search for the nearest neighbors.

DEFINITION 1 (NEAREST NEIGHBOR SEARCH). *Given a set P of polygons and the Jaccard distance metric d , return a set $S \subset P$ such that for some query q , $d(q, s) \leq d(q, p)$ for all $s \in S$ and $p \in P - S$.*

Locality sensitive hashing (LSH) and MinHashing are theoretically robust methods for ANN search [5]. There are many LSH and MinHashing algorithms in the literature for a variety of datasets, such as feature vectors [8], text [5], and images. Polygons represented as variable-length vectors do not lend themselves directly to MinHashing algorithms. To the best of our knowledge, for polygonal shape data, MinHashing algorithms do not exist that directly convert a variable length geometry to area-preserving MinHash signatures that are generally much shorter than original polygons.

Our proposed MinHashing algorithm is based on a variation of rejection sampling proposed earlier for feature vectors [8] and the

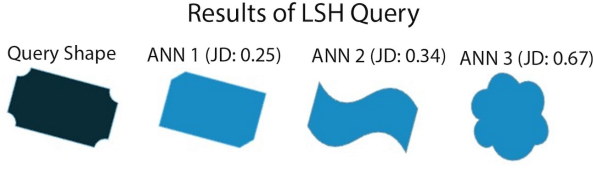


Figure 1: An example of a K-Approximate nearest-neighbor (K-ANN) query and its results for K=3. JD is the Jaccard distance.

Monte Carlo method for statistical sampling. In order to compute the MinHash signature of a polygon, we generate 2D points as samples and test for inclusion/exclusion w.r.t. its area. We have shown that the number of attempts needed by a random 2D point to land inside a polygon in the rejection sampling process preserves the Jaccard similarity when a pair of polygons are compared for approximate similarity. In this work, we do not consider scale and rotational invariance for polygons.

2 Background and Related Work

Many data structures, such as the K-D tree, Voronoi diagram, similarity search tree (SS tree), R-tree, etc., have been designed for distance-based similarity search. Some data structures like the K-D tree and VP-tree (metric-based indexing) support both exact and approximate searches. These trees work effectively for low-dimensional vectors; however, they suffer from the curse of dimensionality when applied to higher-dimensional vectors [6].

Our proposed work is distinct from other related works along three dimensions - 1) type of datasets used for evaluation, 2) distance metric, and 3) theoretical work vs practical system.

Datatype and Distance Metric: Related work on ANN research is based on a variety of datasets such as trajectories [1], curves [2], images [4] and text[5]. The LSH method presented for the trajectories in [1] is not suitable for polygons because the trajectories are open, but the polygons are closed. Therefore, the concept of an area of a polygon is not captured by the LSH methods used for trajectories.

Polygon similarity has been studied in computer graphics and computer vision. First of all, there is a difference in representation, raster vs. vector, which leads to a difference in the design and implementations for the ANN search. Our work considers polygons in vector representation (latitude/longitude list).

3 MinHashing for Approximate Similarity

3.1 Preprocessing

The first step in our PolyMinHash system is to create a global MBR that encloses all polygons, which we then use for rejection sampling. Therefore, we apply two important preprocessing steps. Figure 2 illustrates the PolyMinHash algorithm.

Centering: We shift each polygon so that its centroid (the geometric center) is positioned at the point (0, 0). This makes it easier to compare shapes that might be located far apart on a real-world map, for example, two parks in completely different regions.

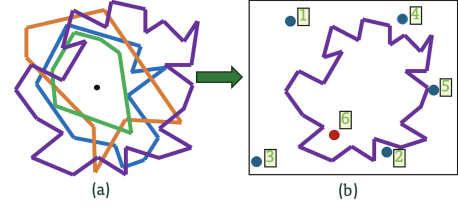


Figure 2: Overview of PolyMinHash system. (a) Center the polygons. (b) Construct the global MBR B and sample 2D points randomly within B for Polygon P. The red dot #6 is the first point that lands inside the polygon. Therefore, the MinHash(P) is 6.

Global minimum bounding rectangle: After centering, we first compute the minimum bounding rectangle (MBR) around each individual polygon. Then, we calculate a global MBR that fully encloses all polygons in the dataset.

3.2 PolyMinHash: MinHashing Scheme for Polygons

Given a polygon P within a global minimum bounding rectangle (MBR) B , we first compute the local MBR of P , $[x_{\min}^P, x_{\max}^P] \times [y_{\min}^P, y_{\max}^P]$. To generate each MinHash value of the signature, we initialize our random number generators using predetermined seeds to ensure consistency across datasets. Fixed seeds make sure that the same sequence of 2D samples will be generated from one run to another. Then we sample a point (x, y) uniformly from the global MBR:

$$x \sim \text{Uniform}(x_{\min}, x_{\max}), \quad y \sim \text{Uniform}(y_{\min}, y_{\max}).$$

DEFINITION 2 (MINHASH FUNCTION). Given a polygon P , we define the MinHash function $h(P)$ as the number of attempts needed by a random 2D point sampled uniformly over the global MBR to land inside P . To generate a MinHash signature of length m , the MinHash function is invoked m times.

By choosing x and y independently and uniformly over their respective intervals, each infinitesimal cell of area $dx dy$ receives the same sampling probability. Once a sample point is generated, several checks are performed sequentially to efficiently filter out points that do not satisfy the following criteria.

MBR check: First, if the sampled point lies outside the local MBR of the polygon, it is immediately rejected without further computation.

Point-in-polygon test: Otherwise a precise point-in-polygon (PnP) test is performed. If the point lies inside the polygon, the number of attempts taken to land a point inside is recorded as a hash value.

The pseudocode of the proposed MinHash scheme is shown in Algorithm 1. This randomized sampling procedure captures the spatial information of a polygonal shape in a concise way.

Algorithm 1 PolyMinHash for Polygon Geometry

Input: Polygon vertex array P , global bounding box B , seed array $\text{seed}[]$, hash length m

Output: Hash vector $h[1..m]$

```

1: Initialize  $h[i] \leftarrow 0$  for all  $i$ 
2: Compute local MBR of  $P$   $\triangleright$  get  $[x_{\min}^P, x_{\max}^P] \times [y_{\min}^P, y_{\max}^P]$ 
3: Initialize Random Number Generators with each value in  $\text{seed}[]$   $\triangleright$  fixed seeds for consistency
4: for  $i \leftarrow 1$  to  $m$  do
5:    $\text{attempts} \leftarrow 0$ 
6:   while true do
7:      $\text{attempts} \leftarrow \text{attempts} + 1$ 
8:      $x \sim \text{Uniform}(x_{\min}, x_{\max})$ 
9:      $y \sim \text{Uniform}(y_{\min}, y_{\max})$ 
10:    if  $x \notin [x_{\min}^P, x_{\max}^P]$  or  $y \notin [y_{\min}^P, y_{\max}^P]$  then
11:      continue  $\triangleright$  outside polygon's local MBR
12:    end if
13:    if point  $(x, y)$  is inside polygon  $P$  then
14:       $h[i] \leftarrow \text{attempts}$ 
15:      break
16:    end if
17:  end while
18: end for
19: return  $h$ 

```

Correctness of PolyMinHash: Here we want to theoretically explore the question - if two polygons are similar, then what are the chances that they will have the same MinHash code. This is an important property to be satisfied in order to hash similar polygons to the same bucket in a hashmap.

THEOREM 1. *For any two polygons P and Q , the probability of a hash collision equals the Jaccard similarity between the two polygons.*

$$\Pr[h(P) = h(Q)] = J(P, Q) \quad (1)$$

where $J(P, Q)$ denotes the Jaccard similarity.

PROOF. Intuitively, the first accepted sample point denoted by variable attempts (line #14) represents a randomly sampled point. The probability that p falls within the area of two similar polygons corresponds to their intersection area (area of overlap).

Let $\{(x_1, y_1), (x_2, y_2), \dots\}$ be the sequence of points independently and uniformly sampled from B . We define the MinHash of polygon P and Q as

$$h(P) = \min\{j \mid (x_j, y_j) \in P\}, \quad h(Q) = \min\{j \mid (x_j, y_j) \in Q\} \quad (2)$$

and let

$$j = \min(h(P), h(Q)). \quad (3)$$

We have the following mutually exclusive events in the PolyMinHash sampling process that generate point samples denoted by $p = (x_j, y_j)$:

- (1) $h(P) = h(Q) = j$ holds if and only if p falls in the intersecting region $P \cap Q$. In this case, the first sampled point to land in either polygon is shared by both, resulting in a hash collision.
- (2) $h(P) > h(Q) = j$ holds if and only if p lands in the area occupied by Q first that does not overlap with P , i.e., p is

contained in the area given by $Q \setminus P$. So, the sampling process will continue for P leading to a higher MinHash value for P .

- (3) $h(Q) > h(P) = j$ holds when p lands in the area occupied by P first that does not overlap with Q , i.e., p is contained in the area given by $P \setminus Q$. So, the sampling process will continue for Q leading to a higher MinHash value for Q .

In the last two cases, hash collision does not occur, which indicates dissimilarity. Since the above-mentioned three cases partition the area denoted by $P \cup Q$, and each point (x_j, y_j) is uniformly sampled over B , the probability of a hash collision is:

$$\begin{aligned} \Pr[h(P) = h(Q)] &= \frac{\text{Area}(P \cap Q)}{\text{Area}(P \cap Q) + \text{Area}(P \setminus Q) + \text{Area}(Q \setminus P)} \\ &= \frac{\text{Area}(P \cap Q)}{\text{Area}(P \cup Q)} = J(P, Q) \end{aligned}$$

□

Using Theorem 1, we can see that probabilistically similar polygons can have the same MinHash codes, which will lead to hash collisions for similar polygons when we insert the polygons into a hashmap. This helps in clustering similar polygons in a hashmap bucket, which helps in the filtering phase of the ANN search.

3.2.1 Running Time and Variance Analysis:

DEFINITION 3. *Let S_p denote the effective sparsity of a polygon P w.r.t. the global MBR:*

$$S_p = \frac{\text{Area of Polygon}}{\text{Area of Global MBR}} \quad (4)$$

This ratio represents the probability that a point sampled uniformly from the global minimum bounding rectangle (MBR) will fall inside the polygon P . This depends on the area and shape of the polygon in relation to the global MBR.

THEOREM 2. *The expected number of randomly sampled points required to land inside the polygon P , denoted by $h(P)$, satisfies:*

$$\mathbb{E}[h(P)] = \frac{1}{S_p} \quad (5)$$

with variance:

$$\text{Var}(h(P)) = \frac{1 - S_p}{S_p^2} \quad (6)$$

4 Experiments

4.1 Data Sets

Our experiments used a variety of real-world polygon datasets, including *Cemetery*, *Sports*, and *Parks* in WKT (Well-Known Text) format, sourced from the UCR STAR website [3]. The characteristics of these datasets are presented in Table 1.

Table 1: Attributes of the dataset.

Name	#Geometries	#queries	Avg. #Pts	FileSize
Urban Areas	11.8K	3K	95.40	24.50 MB
Cemetery	149K	3K	9.26	49.10 MB
Parks	300K	3K	319.00	3750.00 MB
Sports	1M	20K	12.00	511.00 MB

4.2 Experimental evaluation

The experiments were carried out on a multicore CPU system with 128 physical cores, operating at 2 GHz. We utilized 100 MPI processes for all experiments for distributed memory parallelization. We are using performance metrics such as Recall@k and runtime taken by PolyMinHash for querying input polygon compared to the ground truth as found by the Brute-Force (BF). Recall@k is the proportion of top-k approximate nearest neighbors returned by the LSH search, which are part of the set of actual nearest top-k neighbors returned by the BF search. Possible values range from 0 to 1, with higher values indicating better performance.

4.2.1 MinHash Signature Length. We analyze the impact of MinHash length on recall and runtime by varying values from 1 to 5. When MinHash length is short, a large number of polygons tend to be mapped to the same MinHash code, resulting in higher recall, but also increased runtime due to more candidate comparison in the refinement phase, as illustrated in Fig. 3, despite the lower MinHashing time. In contrast, the search process is expedited at the cost of accuracy when the MinHash code is longer, since very few neighbors are assigned to the same m -length MinHash code. Although longer MinHash lengths reduce overall recall, they ensure

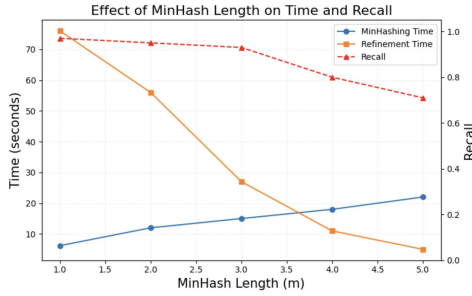


Figure 3: Effect of MinHash length on MinHashing time, Refinement time, and Recall for Cemetery dataset.

that only very close neighbors collide in the same MinHash code, so the candidates that are retrieved are highly similar.

Impact of Different MinHash Lengths on Recall and Runtime in PolyMinHash Across Multiple Datasets. BF: Brute-Force.

Table 2

Dataset	MinHash Length(m)	Recall @10 @50		Time in Seconds			Speedup BF/Total	
				PolyMinHash	BF			
				MinHashing	Refinement	Total		
Cemetery	1	0.97	0.79	6.2	76.1	82.3	216.0	2.6
Cemetery	3	0.93	0.88	16.1	27.5	43.6	216.8	4.9
Cemetery	5	0.71	0.64	26.3	5.0	31.3	217.2	6.9
Urban Area	1	0.91	0.90	3.0	13.1	16.1	32.2	2.0
Urban Area	2	0.81	0.78	4.5	9.7	14.2	31.8	2.2
Urban Area	3	0.76	0.70	6.2	5.7	11.9	31.5	2.6
Sports	1	0.93	0.91	29.1	1696.6	1725.7	4851.8	2.8
Sports	3	0.88	0.85	0.80	1185.9	1269.9	4851.0	3.8
Parks	1	0.93	0.92	0.91	1900.0	1981.8	6291.7	3.1
Parks	2	0.84	0.81	240.1	1260.9	1501.0	6292.6	4.1
Parks	3	0.69	0.66	409.3	734.1	1143.4	6292.7	5.5

4.3 PolyMinHash Filtering Analysis

Figure 4(a) shows how PolyMinHash's filtering stage reduces candidate sets. It shows PolyMinHash prunes over 86% of the dataset with good recall (≥ 0.69) and still prunes around 70% with very good recall (≥ 0.84). Similarly, Fig. 4(b) shows the impact of hash length on pruning. Increasing the number of hash functions m improves the effectiveness of the filter: larger values of m produce higher pruning ratios by generating more discriminative signatures.

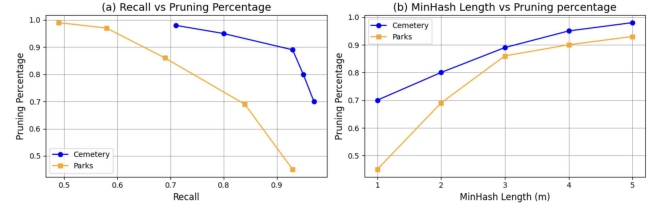


Figure 4: (a) Relation between Recall and Pruning. (b) Effect of MinHash Length on Pruning.

The advantage of PolySS system using 2 hashmaps over an exact similarity-based system is demonstrated in Table 2 with different values for Recall@k.

5 Conclusions

In this work, we have extended the classical MinHash to operate directly on 2D geometric shapes. Our PolyMinHash framework preserves the Jaccard similarity over polygons, saves memory space in representing polygons concisely, and helps in pruning in ANN search.

References

- [1] Maria Astefanoaei, Paul Cesaretti, Panagiota Katsikouli, Mayank Goswami, and Rik Sarkar. 2018. Multi-Resolution Sketches and Locality Sensitive Hashing for Fast Trajectory Processing. In *Proceedings of the 26th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems* (Seattle Washington). ACM, 279–288. doi:10.1145/3274895.3274943
- [2] Anne Driemel and Francesco Silvestri. 2017. Locality-sensitive hashing of curves. CoRR abs/1703.04040 (2017). arXiv:1703.04040 <http://arxiv.org/abs/1703.04040>
- [3] Saheli Ghosh, Tin Vu, Mehrad Amin Eskandari, and Ahmed Eldawy. 2019. UCR-STAR: The UCR Spatio-Temporal Active Repository. *SIGSPATIAL Special* 11, 2 (Dec. 2019), 34–40. doi:10.1145/3377000.3377005
- [4] Kristen Grauman and Trevor Darrell. 2004. Fast contour matching using approximate earth mover's distance. In *Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2004. CVPR 2004.*, Vol. 1. IEEE, I–I.
- [5] Jure Leskovec, Anand Rajaraman, and Jeffrey D Ullman. 2019. *Mining of Massive Datasets* (3 ed.). <http://infolab.stanford.edu/~ullman/mmds/book0n.pdf>
- [6] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *The VLDB Journal* 33, 5 (2024), 1591–1615.
- [7] Karthik Ganesan Pillai, Rafal A Angryk, and Berkay Aydin. 2013. A filter-and-refine approach to mine spatiotemporal co-occurrences. In *Proceedings of the 21st ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. 104–113.
- [8] Anshumali Shrivastava. 2016. Simple and Efficient Weighted Minwise Hashing. In *Advances in Neural Information Processing Systems*, Vol. 29. Curran Associates, Inc.
- [9] Kaibo Wang, Yin Huai, Rubao Lee, Fusheng Wang, Xiaodong Zhang, and Joel H Saltz. 2012. Accelerating pathology image data cross-comparison on cpu-gpu hybrid systems. In *Proceedings of the VLDB endowment international conference on very large data bases*, Vol. 5. NIH Public Access, 1543.